

William Jackson

Senior Software Engineer – React Java (Spring Boot) AWS

Wellington, Nevada, United States | +1 (786) 949-7561 | williamjackson.pro@outlook.com | linkedin.com/in/william-jackson-75b9343b6

PROFESSIONAL SUMMARY

Senior Software Engineer with 10+ years of experience building enterprise platforms across the **financial services, healthcare, retail, and SaaS** industries, specializing in **Java 8–21, Spring Boot 2.x–3.x, React 16–18, TypeScript 4.x–5.x**, distributed APIs, and cloud-native delivery on **AWS**. Strong record of modernizing legacy monoliths into scalable services, resolving production defects, improving release reliability, and delivering secure user-facing products with measurable business impact. Known for leading migrations, stabilizing complex integrations, optimizing data-heavy workflows, and partnering across product, QA, and DevOps to ship maintainable systems that reduce operational friction and accelerate roadmap execution.

SKILLS

- **Frontend: React 16–18, TypeScript 4.x–5.x**, JavaScript ES6+, Redux Toolkit, React Query, Next.js basics, HTML5, CSS3, SCSS, Material UI, Tailwind CSS, responsive UI architecture, reusable component systems, form validation, accessibility, SPA development
- **Backend: Java 8–21, Spring Boot 2.x–3.x**, Spring MVC, Spring Security, Spring Data JPA, Hibernate, REST APIs, microservices, JWT, OAuth2, RBAC, WebSocket integration, batch processing, async workflows
- **Data / Messaging / Search:** PostgreSQL, MySQL, SQL Server, Redis, MongoDB, Kafka, RabbitMQ, AWS SQS/SNS, Elasticsearch, OpenSearch
- **Cloud & DevOps:** AWS, EC2, ECS, EKS, Lambda, API Gateway, S3, RDS, DynamoDB, Docker, Kubernetes, Terraform, Jenkins, GitHub Actions, GitLab CI, Nginx, Linux
- **Observability / Quality / Security:** JUnit, Mockito, Jest, React Testing Library, Cypress, Postman, Swagger / OpenAPI, SonarQube, Prometheus, Grafana, Datadog, ELK, Sentry, IAM, secrets management, secure SDLC

WORK HISTORY

Senior Software Engineer

February 2021 to December 2025

Viaro Networks Inc. - Delaware, United States

- Architected and delivered a multi-tenant event operations platform using **Java 17-21, Spring Boot 3.x, React 18, and TypeScript 5.x**, giving internal teams a faster way to manage event workflows while improving page responsiveness by **34%** across the highest-traffic modules.
- Led migration of legacy backend services from older Spring patterns to **Spring Boot 3.x**, resolving dependency conflicts, Jakarta namespace issues, and broken validation flows that initially caused staging failures during smoke testing.
- Redesigned the frontend component model with **React 18** hooks, shared UI primitives, and stricter state boundaries, which reduced duplicated screen logic and lowered UI-related regression defects by **29%** over successive releases.
- Implemented secure **OAuth2** and **JWT** authorization flows for admin and partner roles, closing permission gaps that had previously allowed inconsistent access behavior in edge-case navigation paths.
- Investigated a persistent production issue where asynchronous event updates were occasionally processed out of order, then introduced idempotent handlers and message validation around **Kafka** consumers to restore consistency in downstream dashboards.
- Built API orchestration services that connected scheduling, ticketing, and analytics workflows, reducing manual coordination effort by **41%** and improving end-to-end reliability for high-volume customer operations.
- Introduced structured observability with **Datadog**, centralized logging, and service-level alerts so the team could isolate release-impacting errors faster and cut mean time to detection during incidents.
- Optimized SQL-heavy reporting endpoints with indexing, pagination redesign, and caching via **Redis**, which improved median report generation times by **46%** for larger enterprise accounts.
- Worked closely with product and design to launch a React-based self-service configuration experience that replaced spreadsheet-driven setup and increased successful first-time customer onboarding by **22%**.
- Strengthened CI/CD pipelines with **GitHub Actions**, Docker-based validation, and contract checks between frontend and backend services, preventing several schema drift issues before they reached production.
- Resolved a recurring memory-pressure problem in containerized services by tuning JVM parameters, tightening object mapping behavior, and reducing unnecessary payload expansion in response builders.

- Mentored engineers on practical migration strategy, code review discipline, and service decomposition, helping the team move features from monolithic modules into cleaner domain-oriented services without delivery disruption

Software Engineer
Avantia, Inc. - Ohio, United States

October 2018 to January 2021

- Built customer-facing application modules using **Java 11**, **Spring Boot 2.x**, **React 17**, and **TypeScript 4.x**, supporting a high-availability SaaS product where reliable dashboard behavior and backend stability were central to daily client operations.
- Delivered RESTful services for account configuration, workflow automation, and reporting, creating a clearer domain separation that improved maintainability and reduced backend change collisions across squads.
- Refactored a brittle server-rendered workflow into a modern **React** interface with reusable forms, validation layers, and API-driven state handling, which increased task completion efficiency by **27%** for operations users.
- Diagnosed and fixed an intermittent concurrency defect in scheduled jobs where overlapping executions generated duplicate outbound notifications, then applied locking and retry-safe logic to eliminate repeated processing.
- Migrated legacy authentication code toward **Spring Security** standards and modern token handling, closing several long-standing edge-case session issues that had caused inconsistent logout and token refresh behavior.
- Introduced **RabbitMQ**-based asynchronous processing for non-blocking background tasks, reducing synchronous request pressure and cutting peak-time API latency by **31%**.
- Improved frontend resilience by introducing better error boundaries, loading states, and retry messaging, which lowered support complaints tied to partial API failures and improved user trust in unstable network conditions.
- Collaborated with QA to reproduce a difficult serialization bug tied to nested payloads and null handling, then fixed mapper behavior and request contracts to prevent malformed API responses under rare but critical conditions.
- Expanded reporting functionality with export, filtering, and saved-view capabilities, providing business users with more flexible access to operational data while reducing manual data preparation work.
- Containerized core services with **Docker** and standardized deployment scripts, creating more predictable lower-environment behavior and reducing environment-specific defects that previously slowed release cycles.
- Helped establish better pull request practices, API documentation, and integration testing so cross-team feature delivery became more reliable even as the product surface area grew.

Software Developer
ArnAmy, Inc. - Florida, United States

March 2016 to September 2018

- Developed internal and external product features using **Java 8–11**, **Spring Boot 2.x**, **React 16–17**, and PostgreSQL, contributing to a business platform that required dependable transaction handling and responsive browser-based workflows.
- Implemented modular REST endpoints and service-layer abstractions that simplified future feature expansion, especially for customer records, approval flows, and document-centric business processes.
- Resolved a difficult production issue where large payload submissions occasionally timed out under peak load, then improved request chunking, query efficiency, and server-side processing boundaries to stabilize the workflow.
- Supported migration away from tightly coupled legacy modules by extracting reusable services and shared DTO patterns, making it easier for teams to build new capabilities without repeatedly touching fragile older code.
- Reworked several dense frontend screens into maintainable **React** components with cleaner prop contracts and predictable state handling, reducing defect frequency and improving handoff quality between engineers.
- Added **Redis** caching to high-read endpoints and optimized SQL execution plans, which improved response times by **38%** for common dashboard and lookup actions used throughout the day.
- Partnered with product managers to implement approval-routing enhancements and audit visibility features that gave users more transparency into state changes and reduced support escalation volume by **18%**.
- Fixed a recurring data mismatch issue caused by inconsistent timezone handling between UI filters and backend queries, then standardized serialization rules so reports aligned correctly across user regions.
- Improved build and release confidence by adding unit tests, integration coverage, and API validation checkpoints that caught regressions earlier and reduced rework after QA handoff.

- Contributed to deployment automation and environment configuration cleanup, helping the team move from inconsistent manual release habits toward more repeatable delivery practices.

Junior Software Developer
Centric Tech Inc. - New Jersey, United States

October 2014 to February 2016

- Contributed to enterprise application development with **Java 8**, Spring-based services, server-rendered web modules, and early **React 16** adoption, building a strong engineering foundation in API design, business logic implementation, and defect resolution.
- Enhanced customer management and administrative workflows by implementing backend rules, validations, and UI improvements that reduced manual corrections and improved operational accuracy for internal teams.
- Investigated a frustrating issue where partially completed form data was lost after session interruptions, then introduced safer draft persistence and recovery behavior that improved completion reliability for business users.
- Supported migration of selected frontend modules from older page-centric implementations into reusable React-driven interfaces, which made future enhancement work more flexible and reduced duplication in shared screens.
- Improved batch processing jobs by tracing memory spikes and unoptimized record iteration, then restructuring processing logic so long-running tasks completed more consistently during overnight execution windows.
- Fixed API response inconsistencies caused by mixed date formatting and weak null handling, which had triggered avoidable client-side errors in downstream screens and reports.
- Worked with senior engineers to strengthen SQL queries, indexing patterns, and transaction handling, helping reduce slow-screen complaints by **24%** in data-heavy internal workflows.
- Added logging, validation checks, and clearer failure messages around integration points so support teams could identify real causes faster instead of manually reproducing issues from incomplete error output.
- Participated in release support, bug triage, and environment verification efforts, building practical discipline around regression analysis, reproducibility, and safe incremental delivery.

EDUCATION

Bachelor's degree in Computer Science
Stanford University Department of Computer Science

2010 to 2014

- Built strong foundations in **algorithms**, **databases**, and **software engineering**, applying coursework through practical team projects and production-style system design.

CERTIFICATIONS

- **AWS Certified Developer – Associate** — 2024
Validated hands-on ability to build, deploy, and troubleshoot cloud-native applications on AWS with strong focus on scalable services, security, and operational reliability.
- **Oracle Certified Professional: Java SE Developer** — 2021
Demonstrated strong understanding of core **Java** development concepts, object-oriented design, collections, concurrency fundamentals, and maintainable application structure.
- **Professional Scrum Master I** — 2023
Confirmed practical knowledge of Agile delivery, cross-functional collaboration, and iterative engineering execution for teams building production software in fast-moving environments.